

Plan Wdrożenia Systemu TaskAlert

ARCHITEKTURA ROZWIĄZAŃ KORPORACYJNYCH I MODUŁÓW ALERTOWYCH (FIREBASE & GITHUB PAGES)

Wymaganie projektowe v3.0: Niniejszy dokument stanowi oficjalną rewizję planu inżynierskiego systemu **TaskAlert**. Wszystkie komponenty zostały dostosowane do obsługi zaawansowanych procesów przypomnień o charakterze operacyjnym i prawnym (Samochody, Kadry, Alerty e-mailowe z wyprzedzeniem czasowym) w modelu bezserwerowym.

1. Specyfikacja Funkcjonalna i Biznesowa Modułów

Agent piszący kod ma za zadanie zaimplementować trzy bazowe szablony alertów oraz system elastycznego zarządzania strukturą klasyfikacji:

- **Moduł Samochody:** Automatyzacja powiadomień i ewidencji dla **polis ubezpieczeniowych (OC/AC)** oraz **przeglądów technicznych** pojazdów firmowych/prywatnych.
- **Moduł Kadry:** Nadzór nad ciągłością uprawnień pracowników, obejmujący obowiązkowe okresowe **badania lekarskie** oraz **szkolenia BHP**.
- **Moduł Inne:** Elastyczny koszyk zadań dla nieklasyfikowanych zdarzeń terminowych.
- **Zarządzanie Kategoriami:** Domyślny słownik systemowy inicjalizuje kategorie: kadry, samochody oraz inne. Interfejs użytkownika musi posiadać dedykowany widok CRUD umożliwiający pełną **edycję, dodawanie nowych pozycji oraz modyfikację** nazw kategorii.
- **Interfejs Dodawania Pozycji:** Formularz dynamiczny pozwalający na wprowadzanie nowych rekordów alertowych w ramach wybranych kategorii.

2. Architektura Powiadomień i Logika Biznesowa Zdarzeń

System realizuje wieloetapowy proces eskalacji alertów, porzucając standardowe powiadomienia ad-hoc na rzecz ustrukturyzowanych powiadomień e-mailowych:

1. **Reguła Wyprzedzenia Czasowego (Dual-Window Alert):** Silnik powiadomień musi wysłać wiadomość e-mail w dwóch predefiniowanych oknach czasowych:
 - **Dokładnie 1 miesiąc (30 dni) przed** datą wygaśnięcia wpisu.
 - **Dokładnie 2 tygodnie (14 dni) przed** datą wygaśnięcia wpisu.
2. **Logika Zamknięcia i Rekurencji (Wprowadzenie Wykonania i Następnej Daty):** W momencie odznaczenia alertu jako "wykonany", system wymaga od użytkownika wprowadzenia daty faktycznego wykonania. Na podstawie reguły interwału, system automatycznie wylicza, generuje oraz zapisuje w bazie **następną datę wygaśnięcia (kolejną instancję przypomnienia)**.
3. **Przekierowanie i Korespondencja Zastępcza:** Oprócz domyślnego adresu e-mail powiązanego z kontem użytkownika, formularz dodawania pozycji zawiera opcjonalne pole na **drugi adres e-mail (przekierowanie/kopia)**. W przypadku jego uzupełnienia, powiadomienia są wysyłane równolegle na oba adresy.

3. Rozszerzenie Schematu Bazy Danych (Cloud Firestore)

W strukturze dokumentu w kolekcji `/users/{uid}/reminders/{reminderId}` należy wdrożyć następujące pola techniczne:

Pole w Firestore	Typ danych	Wymaganie Walidacyjne / Zastosowanie
categoryRef	String	Identyfikator kategorii (powiązany z dynamiczną kolekcją /categories z opcją edycji).
subType	String	Wartości strukturalne: polisa przeglad badania_lekarskie bhp custom.
primaryEmail	String	Główny adres e-mail do wysyłki powiadomień.
secondaryEmail	String	Drugi adres e-mail do przekierowania powiadomień (opcjonalny).
expiryDate	Timestamp	Data końcowa ważności dokumentu/badania/polisy. Target dla schedulerów.
lastExecutedAt	Timestamp	Data ostatniego faktycznego wykonania wprowadzona przez operatora.
alertFlags	Map	Flagi zapobiegające dublowaniu maili: {monthSent: Boolean, twoWeeksSent: Boolean}.

4. Implementacja Silnika Powiadomień (Firebase Cloud Functions & Cloud Tasks)

Ponieważ statyczny hosting na GitHub Pages uniemożliwia ciągłe uruchamianie zadań cron, mechanizm powiadomień e-mail opiera się na dobowej funkcji uruchamianej przez **Firebase Cloud Scheduled Functions** zintegrowanej z dostawcą SMTP (np. SendGrid lub Firebase Extension: *Trigger Email from Firestore*).

Kod realizujący sprawdzanie okresów 30-dniowych oraz 14-dniowych dla agenta programistycznego (functions / src/alerts.ts):

```
import * as functions from 'firebase-functions';
import * as admin from 'firebase-admin';

export const dailyAlertScheduler = functions.pubsub
  .schedule('every 24 hours')
  .onRun(async (context) => {
    const db = admin.firestore();
    const now = new Date();

    const targetMonth = new Date(now.getTime() + 30 * 24 * 60 * 60 * 1000);
    const targetTwoWeeks = new Date(now.getTime() + 14 * 24 * 60 * 60 * 1000);

    // Pobranie aktywnych alertów z bazy danych TaskAlert
    const snapshot = await db.collectionGroup('reminders')
      .where('status', '==', 'active')
      .get();

    for (const doc of snapshot.docs) {
      const data = doc.data();
      const expiry = data.expiryDate.toDate();

      // Porównanie dat i wysyłka na podstawowy oraz drugi adres email (przekierowanie)
      if (isSameDay(expiry, targetMonth) && !data.alertFlags?.monthSent) {
        await sendAlertEmail(data.primaryEmail, data.secondaryEmail, data, '1 miesiąc');
        await doc.ref.update({ 'alertFlags.monthSent': true });
      }

      if (isSameDay(expiry, targetTwoWeeks) && !data.alertFlags?.twoWeeksSent) {
        await sendAlertEmail(data.primaryEmail, data.secondaryEmail, data, '2 tygodnie');
        await doc.ref.update({ 'alertFlags.twoWeeksSent': true });
      }
    }
  })
```

```
});  
  
function isSameDay(d1: Date, d2: Date): boolean {  
  return d1.toISOString().split('T')[0] === d2.toISOString().split('T')[0];  
}
```

5. Przepływ UI dla Agenta Kodującego (Flutter Web / PWA)

1. **Widok Edycji Kategorii:** Zaimplementuj komponent `CategoryManagerGrid`. Pozwala on na zmianę etykiet domyślnych (np. zmiana "Kadry" na "HR i Płace") oraz dynamiczne wywołanie metody `Firestore.collection('categories').add()` w celu rozbudowy struktury.
2. **Wprowadzanie Wykonania i Następnej Daty:** Przyciski interakcji z powiadomieniem w aplikacji **TaskAlert** otwierają okno dialogowe `ExecutionDialog`. Pola wejściowe: Faktyczna data wykonania (`DatePicker`) oraz Następna wymagana data (`Calculated/Manual DatePicker`). Zapisanie formularza aktualizuje obiekt w bazie danych i resetuje mapę `alertFlags` do stanu `false``, przygotowując system na kolejny cykl powiadomień.